



Chapter 3

Types and Interfaces

Concepts and Principles: Class diagram, coupling, extensibility, function object, interface, Interface Segregation Principle, interface type, iterator, polymorphism, reusability, separation of concerns, specification, subtyping;

Patterns and Antipatterns: ITERATOR, STRATEGY, SWITCH STATEMENT[†].

In the previous chapter we saw how to define well-encapsulated classes, but conveniently left out the question of how objects of these classes would interact. We now start addressing this question. Interactions between objects are mediated through interfaces. The term *interface* is overloaded in programming: it can have different meanings depending on the context.

Design Context

The examples in this chapter concern the design of a class library to allow client code to instantiate and use a deck and other collections of card objects to support the development of card games.

3.1 Decoupling Behavior from Implementation

An interface to a class consists of the methods of that class that are accessible (or visible) to another class. What methods are accessible depends on programming language rules that take into account access modifiers and scopes (see Section 2.3). For now, we define **the interface to a class as the set of its public methods** (I will extend this definition in Chapter 7). Let us consider the following code:

```

public class Client {
    private Deck aDeck = new Deck();
}

public class Deck {
    public void shuffle() { ... }
    public Card draw() { ... }
    public boolean isEmpty() { ... }
}

```

The interface of class `Deck` consists of three methods. The code in other classes can interact with objects of class `Deck` by calling these and only these methods. Here we would say that the interface of class `Deck` is fused, or *coupled*, with the class definition. In other words, the interface of class `Deck` is just a consequence of how we defined class `Deck`: there is **no way to get the three services that correspond to the three methods of the class, without interacting with an instance of class `Deck`**. In our example, to shuffle the deck, client code will need to invoke method `shuffle()` of a field or local variable of type `Deck`: there is no other option.

There can be, however, situations in which we may want to *decouple* the interface of a class from its implementation. These are situations in which **we want to design the system so that one part of the code can depend on the availability of a service, without being tied to the exact details of how this service is implemented**. Given that we are designing a library that can be used to build different card games, we note that many card games require the user to draw cards, but not necessarily from a standard deck of 52 cards. For example, some games might require drawing cards from an aggregation of multiple decks of cards, from a set of cards of only one suit, from ordered sequences of cards, etc. Let us consider the following code that draws cards from a deck up to a required number.

```

public static List<Card> drawCards(Deck pDeck, int pNumber) {
    List<Card> result = new ArrayList<>();
    for (int i = 0; i < pNumber && !pDeck.isEmpty(); i++) {
        result.add(pDeck.draw());
    }
    return result;
}

```

This method can only be used with sequences of cards that are an instance of class `Deck`. This is a pity, because **exactly the same code could be useful for any object that has the two required methods `draw()` and `isEmpty()`**. Here it would be useful to **specify an abstraction of an interface without tying it to a specific class. This is where Java interface types come in**. In Java, interface types provide a *specification* of the methods that it should be possible to invoke on an object. **With interface types, we can define an abstraction `CardSource` as any object that supports a `draw()` method and an `isEmpty()` method:**

```
public interface CardSource {
    /**
     * Returns a card from the source.
     *
     * @return The next available card.
     * @pre !isEmpty()
     */
    Card draw();

    /**
     * @return true if there is no card in the source.
     */
    boolean isEmpty();
}
```

This interface declaration¹ lists two methods, and includes comments that specify the behavior of each method. The specification of `draw()` includes the precondition that the method can only be invoked if `isEmpty()` is false. This precondition is provided to support the use of design by contract (see Section 2.9), and takes into account the existing state of the object. Because **interface method declarations are a specification and not an implementation**, details of what the method is expected to perform are very important. With a method implementation, it could always be possible to inspect the code (if we have access to it) and infer the specification. This is not an ideal situation, but it is better than nothing. With interface methods, though, reverse-engineering what the method does is not possible. In Java terminology, **methods that do not have an implementation are called abstract methods**.² To tie a class with an interface, we use the `implements` keyword.

```
public class Deck implements CardSource {
    ...
}
```

The `implements` keyword has two related effects:

- It provides a guarantee that instances of the class type will **have concrete implementations for all the methods in the interface type**. This guarantee is enforced by the compiler.
- It creates a **subtype relationship between the implementing class and the interface type**: here we can now say that a `Deck` is a type of `CardSource`.

The subtype relation between a concrete class and an interface is what enables the use of *polymorphism*, namely, the ability to have different shapes. Here, **`CardSource` is an abstraction that can present itself in different concrete shapes**. Each concrete shape corresponds to a different implementation of the `CardSource` interface.

¹ There is an important distinction between the general concept of an *interface*, and the specific *interface* construct in Java. When the difference is clear from the context, I simply use the term *interface*. When necessary, I use the expression *interface type* to refer to the Java construct.

² Prior to Java 8, all interface methods were automatically abstract. With Java 8, this is no longer true, because interfaces can include default and static methods, which have an implementation.

For polymorphism to be useful, it is important to remember that according to the rules of the Java type system, it is possible to assign a value to a variable if the value is of the same type or a *subtype* of the type of the variable. Because the interface implementation relation defines a subtype relation, references to objects of concrete classes declared to implement an interface can be assigned to variables declared to be of the interface type. For example, because class `Deck` declares to implement interface `CardSource`, we can assign a reference to an object of class `Deck` to a variable of type `CardSource`, as such:

```
CardSource source = new Deck();
```

Taking this idea further, this means we can make our implementation of the `drawCards` method much more *reusable*:

```
public static List<Card> drawCards(CardSource pSource, int pNum) {
    List<Card> result = new ArrayList<>();
    for (int i = 0; i < pNum && !pSource.isEmpty(); i++) {
        result.add(pSource.draw());
    }
    return result;
}
```

The method is now applicable to objects of any class that implements the `CardSource` interface.

Another illustration of the use of polymorphism is the use of concrete vs. abstract types in the Java Collections Framework.

```
List<String> list = new ArrayList<>();
```

`List<T>` is an interface that specifies the usual services (add, remove, etc.), and `ArrayList` is an implementation of this service that uses an array.³ But we can replace `ArrayList` with `LinkedList` and the code will still compile. Even though the details of the list implementation differ between `ArrayList` and `LinkedList`, **they both provide the methods required by the `List` interface**, so it is permissible to swap them. Polymorphism provides two useful benefits in software design:

- *Loose coupling*, because the **code using a set of methods is not tied to a specific implementation of these methods**.
- *Extensibility*, because we can easily add new implementations of an interface (new “shapes” in the polymorphic relation).

3.2 Specifying Behavior with Interface Types

Typical design questions related to interfaces include: *do I need a separate interface?* and *what should this interface specify?* There are no universal answers to

³ As indicated by the type parameter `<T>`, interface `List<T>` is a *generic type* (see Section A.6 in the appendix). To simplify the presentation, I omit the type parameter when referring to types within the text when it is clear from the context that a type is a generic type.

such questions, because in each case the task is to determine if interfaces can help us solve a design problem or realize a particular feature. One good illustration of both the purpose and usefulness of interfaces in Java is the `Comparable<T>` interface.

One obvious task to be implemented in the `Deck` class is to shuffle a deck of cards. This can be realized trivially with the help of a library method.

```
public class Deck {
    private List<Card> aCards = new ArrayList<>();

    public void shuffle() {
        Collections.shuffle(aCards);
    }
}
```

As its name implies, the `shuffle` library method randomly reorders the objects in the argument collection. This is an example of code reuse because it is possible to reuse the library method to reorder any kind of collection. Here reuse is easy because to shuffle a collection, we do not need to know anything about the items being shuffled.

But what if we want to reuse code to *sort* the cards in the deck? Sorting, like many classic computing problems, is supported by many existing quality implementations. In most software development situations, it would not be worthwhile to hand-craft one's own sorting algorithm. The Java `Collections` class conveniently supplies us with a number of sorting functions. However, if we opportunistically try the following without further consideration:

```
List<Card> cards = ...;
Collections.sort(cards);
```

we are rewarded with a possibly cryptic compilation error.⁴ This should not be surprising, though, because how exactly is a library method supposed to know how we want to sort our cards? Not only is it impossible for the designers of library methods to anticipate all the user-defined types that can be written, but even for a given type like `Card`, different orderings are possible (e.g., by rank, then suit, or vice-versa).

The `Comparable<T>` interface helps solve this problem by defining a piece of behavior related specifically to the comparison of objects, in the form of a single `compareTo(T)` abstract method. The specification for this method is that **it should return 0 if the implicit argument is equal to the explicit argument, a negative integer if it should come before, and a positive integer if it should come after.** Given the existence of this interface, the internal implementation of `Collections.sort` can now rely on it to compare the objects it should sort. Conceptually, the internal code of the `sort` implementation looks a bit like this:

```
if (object1.compareTo(object2) > 0) ...
```

So, from the point of view of the implementation of `sort`, it really does not matter what the object is, as long as it is *comparable* with another object. This is a great

⁴ The method `sort(List<T>)` in the type `Collections` is not applicable for the arguments (`List<Card>`). Results can vary on different compilers.

example of how interfaces and polymorphism support loose coupling: the code of `sort` depends on the minimum possible piece of functionality required from its argument objects. This is a good general insight on how to define interface types. Ideally, they should capture the smallest cohesive slice of behavior that is expected to be used by client code. For this reason, **many interface types in Java are named with an adjective that ends in *-able*, a suffix that means *fit to be*.... Besides `Comparable`, examples include `Iterable`, `Serializable` and `Cloneable`.**

To make it possible for us to sort a list of cards, we therefore have to provide this *comparable* behavior and declare it with the `implements` keyword:

```
public class Card implements Comparable<Card> {
    public int compareTo(Card pCard) {
        return aRank.ordinal() - pCard.aRank.ordinal();
    }
}
```

This minimal implementation sorts cards by ascending rank, but leaves the order of suits undefined, which leads to unpredictability. A more useful implementation of the `Comparable` interface would provide a well-defined total ordering.

Because Java interfaces are types, the type-checking mechanism that is part of the compilation process makes it possible to detect that a `List<Card>` object cannot be passed to `Collections.sort` unless the `Card` class declares to implement the `Comparable<Card>` interface. How this happens is outside the scope of this book because it requires a good understanding of the typing rules for Java generic types (see Section A.6 in the appendix).

Many other library types that have a so-called *natural* ordering implement the `Comparable` interface. This includes `String` (with lexicographic order) but also many other pervasive types. In particular, Java enumerated types implement `Comparable` by comparing instances of an enumerated type according to their ordinal value. With this knowledge in hand, we observe that our implementation of `Card.compareTo`, above, actually re-implements reusable behavior provided by the enumerated types. We thus have an opportunity to simplify our code:

```
public class Card implements Comparable<Card> {
    public int compareTo(Card pCard) {
        return aRank.compareTo(pCard.aRank);
    }
}
```

Using small interfaces encourages the respect of a software design principle called *separation of concerns*. The idea of separation of concerns is that one abstraction should map to a single *concern* (or area of interest) for developers. In designs that do a poor job at separation of concerns, we find concerns to be *tangled* within an abstraction (a method for example), and/or *scattered* across multiple different abstractions. Here the use of the `Comparable` interface is a good example of effective separation of concerns: the code to compare cards is entirely contained within a clearly defined and identified abstraction (the `compareTo` method), which does only one thing.

3.3 Class Diagrams

Designs where the important concerns revolve around the definition of types and relations between types can become overwhelming to describe in code, and are more easily captured through a diagram. Class diagrams represent a *static*, or *compile-time*, view of a software system. They are useful to represent how types are defined and related, but are a poor vehicle for capturing any kind of *run-time* property of the code. Class diagrams are the type of UML diagrams that are the closest to the code. However, it is important to remember that the point of UML diagrams is not to be an exact translation of the code. As models, they are useful to capture the essence of one or more design decisions without having to include all the details.

Class diagram have an extensive associated notation. In a class diagram, there is typically more going on than, say, in an object diagram. I only use a subset of the notation in this book. The Further Reading section includes references for UML class diagrams. Figure 3.1 shows the main concepts used in this book. In the figure, all quotes are taken from *The Unified Modeling Language Reference Manual, 2nd edition* [15]. The interpretation of the concepts of aggregation, association, and dependency will become clearer as we progress through the chapters. For now, it is sufficient to know that these concepts represent that two classes are somehow related. The concept of navigability, represented with an arrow head, models how code supports going from objects of one type to objects of another type. Navigability can be unidirectional (as shown), bidirectional, or unspecified.

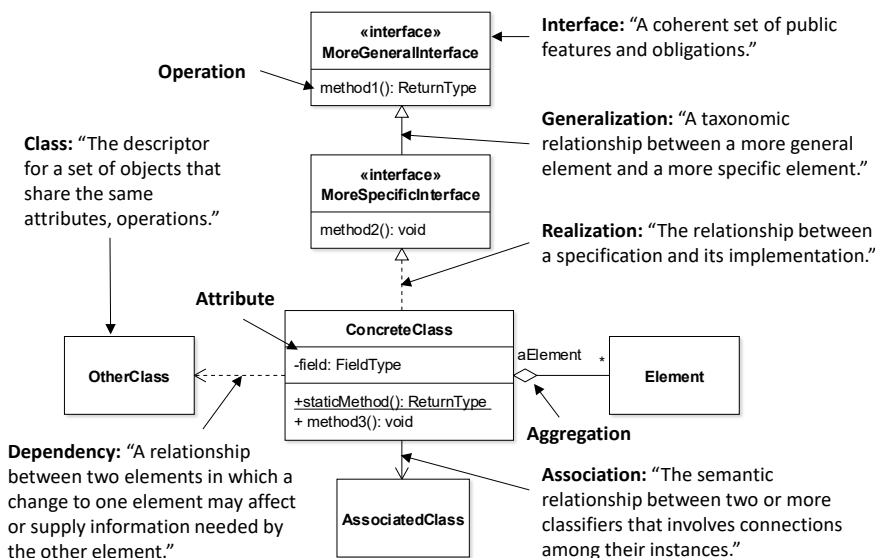


Fig. 3.1 Selected notation for class diagrams

Figure 3.2 shows an example of a class diagram that models some of the key relations between the design elements for a card game that we have seen so far. We can observe the following:

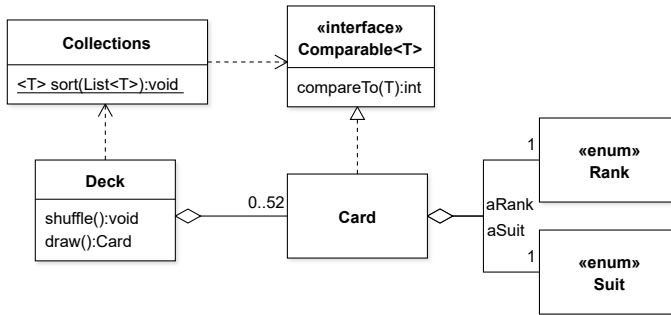


Fig. 3.2 Sample class diagram showing key decisions in the design of a `Deck` class

- The box that represents class `Card` *does not have attributes* for `aRank` and `Suit` because these are represented as aggregations to `Rank` and `Suit` enumerated types, respectively. It is a modeling error to have *both* an attribute *and* an aggregation to represent a given field.
- The methods of class `Card` are not represented. Because they are just the constructor and accessors, I judged this to not be very insightful information. It would not be wrong to include them, but it might clutter the diagram.
- In the UML, there is no good way to indicate that a class *does not have* a certain member (field or method). To convey the information that `Card` *does not have* setters for the two fields, it would be necessary to include this using a note.
- Representing generic types is a bit problematic, because in some cases it makes more sense to represent the type parameter (`Comparable<T>`) and in some other cases it makes more sense to represent the type argument (`Comparable<Card>`). In this diagram I went with the type parameter because I wanted to show how `Collections` depends on `Comparable` in general.
- To indicate that a class member (field or method) is static, we underline it.
- The model includes cardinalities to indicate, for example, that a deck instance will aggregate between zero and 52 instances of `Card`. Typical values for an association's cardinality include a specific number (for example, 1), the wildcard `*` (which means zero or more), and ranges such as `M..N` (which means between `M` and `N`, inclusively).

3.4 Function Objects

In practice, an interface type often defines only a subset of the operations of the classes that implement it. This scenario is exemplified by `Comparable`: the complete implementation of `Card` comprises methods, such as `suit()` and `rank()`, that add to the slice of behavior required by the `Comparable` interface. There are other situations, however, where it is convenient to define classes that specialize in implementing *only* the behavior required by a small interface with only one method.

Let us continue with the problem of comparing cards. Implementing the `Comparable` interface allows instances of `Card` to compare themselves with other instances of `Card` using one strategy, for example, by comparing the card's rank, and using suits to break ties. What if we are designing a game where we need to sort cards according to different strategies, and occasionally switch between them? One could tweak the code of `compareTo`, for instance by relying on a global variable that stores the required strategy and switching the comparison strategy based on this flag. However, harebrained schemes of this nature have many drawbacks. In our case, using such a flag variable would degrade the separation of concerns between representing a card and knowledge of how the card should be sorted, and generally make the code harder to understand.

In fact, the use of this kind of switching is considered a design antipattern called `SWITCH STATEMENT`.⁵ A more promising solution is to move the comparison code to a separate object. This solution is supported by the `Comparator<T>` interface. The abstract method in this interface is `compare` (as opposed to `compareTo`).⁶

```
int compare(T pObject1, T pObject2)
```

As for `Comparable`, `Comparator` is a generic type, so in the above declaration, `T` refers to a type parameter that must be replaced by a concrete type (for example, type `Card`). The most notable difference between method `compare` of interface `Comparator` and method `compareTo` of interface `Comparable` is that `Comparator#compare` takes two arguments instead of one. Indeed, its specification is very similar to that of `Comparable#compareTo`, except that instead of comparing the implicit parameter (the `this` object) with an explicit parameter, it compares two explicit parameters with each other. Not surprisingly, library methods were also designed to work with this interface. For example: `Collections.sort`:

```
Collections.sort(List<T> list, Comparator<? super T> c)
```

This method can sort a list of objects that do not necessarily implement the `Comparable` interface, by taking as argument an object guaranteed to be able to compare two instances of the items in the list. One can now define a *rank first* comparator:

⁵ There is evidence of the antipattern whether or not an actual `switch` statement is used, because the latter can be emulated through `if-else` statements.

⁶ As of Java 8, this interface defines an intimidating list of methods. These are not important here. In Chapter 9 I revisit this interface to explain how we can leverage some of the additional methods.

```
public class RankFirstComparator implements Comparator<Card> {
    public int compare(Card pCard1, Card pCard2) {
        /* Comparison code */
    }
}
```

and another *suit first* comparator:

```
public class SuitFirstComparator implements Comparator<Card> {
    public int compare(Card pCard1, Card pCard2) {
        /* Comparison code */
    }
}
```

and sort with the desired comparator:⁷

```
Collections.sort(aCards, new RankFirstComparator());
```

In this scenario, an instance of `Comparator` is an object that only provides the implementation for a single method. Such objects are referred to as *function objects*. Their interface typically maps one-to-one to that of an interface type.

The use of comparators (and similar function objects) introduces many interesting design questions and trade-offs. First, if comparator classes are defined as standalone top-level Java classes, the code of their `compare` method will not have access to the private members of the objects they compare. In some cases the information available from getter methods is sufficient to implement the comparison, but in other cases implementing the `compare` method will require access to private members.

In such cases, one option to give comparator classes access to private members of the classes they compare is to declare the comparator classes as *nested classes* (see Section 4.9) of the class that defines the objects being compared:

```
public class Card {
    static class CompareBySuitFirst implements Comparator<Card> {
        public int compare(Card pCard1, Card pCard2) {
            /* Comparison code */
        }
    }
}
```

To client code, the impact of this change in design is minimal: the only difference is the additional qualification of the name of the comparator class:

```
Collections.sort(aCards, new Card.CompareBySuitFirst());
```

Another option is to define comparator classes as *anonymous classes*. In cases where the comparator is only referred to once, this makes a lot of sense:

⁷ As of Java 8, method `sort` is also available on the `Collection` (no 's') interface. However, to preserve the symmetry with the use of the `Comparable<T>` interface, I retain this version.

```

public class Deck {
    public void sort() {
        Collections.sort(aCards, new Comparator<Card>() {
            public int compare(Card pCard1, Card pCard2) {
                /* Comparison code */
            }
        });
    }
}

```

A third option is to use a *lambda expression*. Lambda expressions are a form of anonymous functions. These and related mechanisms form the topic of Chapter 9. However, since it is possible to implement a very close equivalent to the code above using a lambda expression, I provide it here. The basic idea is based on the observation that to supply a comparator, not only do we not need to name a class because it is only an implementation of the `Comparator` interface, we also do not need to name the method, because it is only an implementation of `compare`. How this code actually works is explained in Chapter 9.

```

public class Deck {
    public void sort() {
        Collections.sort(aCards, (card1, card2) ->
            card1.rank().compareTo(card2.rank()));
    }
}

```

In the two examples above, we have brought back the problem of encapsulation, because the code in the anonymous class that implements the comparison is defined outside of the `Card` class. We can solve this with the help of a static *factory method*. The term *factory method* refers to methods whose primary role is to create and return an object.

```

public class Card {
    public static Comparator<Card> createByRankComparator() {
        return new Comparator<Card>() {
            public int compare(Card pCard1, Card pCard2) {
                /* Comparison code */
            }
        };
    }
}

```

A final question is whether a comparator should store data. For example, instead of having different comparators for sorting cards by rank and suit, we could define a `UniversalComparator` that has a field of an enumerated type that stores the desired type of comparison. Although this solution is workable, it can lead to code that is harder to understand, for reasons explained in Section 3.7 and further discussed in Chapter 4.

3.5 Iterators

A common requirement when designing a data structure is to gain access to a collection of encapsulated objects without violating the principle of information hiding. For example, if we are designing a type to represent a deck of cards, we may need to give client code access to the cards in the deck. This problem was originally introduced in Section 2.7, where the solution proposed was to return copies of the internal data. For example, to return a copy of the list of cards encapsulated within a `Deck` instance. One issue with this solution is that it can subtly leak information about the way a structure is stored internally (or at least, give the impression that it leaks this information). For example, if we choose to return a deck's cards as a `List`:

```
public List<Card> cards() { ... }
```

code using the `Deck` may start relying on the operations defined on a list, or make the assumption that cards are internally stored in a list within a `Deck`. To achieve an even higher level of information hiding, it would be better to allow client code access to the internal objects of another object, without exposing anything about the internal structure of the encapsulating object. This design feature is supported by the concept of an *iterator*. The concept of an iterator is very general, and iterators are employed in many programming languages.

In Java, iterators are easy to use, but understanding how they work requires being aware of a careful coordination between at least three types of objects. Iterators also provide an example of the use of interfaces types and polymorphism.

To support iteration we must first have a specification of what it means to iterate. As usual, this specification is captured in an interface: in this case the `Iterator<T>` interface. This interface defines two abstract methods: `hasNext()` and `next()`. So, according to the rules of subtyping, once a piece of code gains access to a reference to an object of any subtype of `Iterator`, the client code can iterate over it, independently of what the actual class of the object is. To enable iteration over the cards of a `Deck`, let us simply redefine the `cards` method to return an iterator instead of a list:

```
public Iterator<Card> cards() { ... }
```

This way, to print all the cards in a deck, we can do:

```
Iterator<Card> iterator = deck.cards();
while (iterator.hasNext()) {
    System.out.println(iterator.next());
}
```

Although this design achieves our decoupling goal, we can generalize it to great effect. A first important insight is that in most software systems there will be different types of objects that it would be useful to iterate over. Lists are an obvious example. In our case we also have a `Deck`. The issue with the iterator system as we have it now, though, is that different classes define a different way to obtain an iterator. For class `List`, it is through the method `iterator()`. For our `Deck`

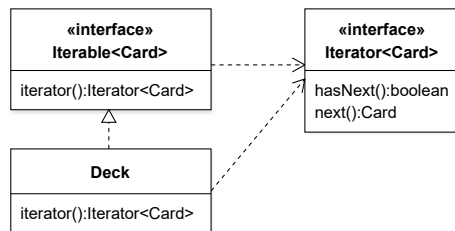
class, it is through method `cards()`. Although the behavior in both cases is identical (return an iterator), the *name* of the service is different. We can solve this issue with another interface. The `Iterable<T>` interface specifies the smallest slice of behavior necessary to make it possible to iterate over an object. To be able to polymorphically iterate over an object, the only thing we need from this object is that it supplies an iterator. So the only abstract method of the `Iterable<T>` interface is `Iterator<T> iterator()`.

We can make our `Deck` class iterable by implementing the `Iterable<Card>` interface and renaming the `cards()` method to `iterator()`:

```
public class Deck implements Iterable<Card> {
    public Iterator<Card> iterator() {
        ...
    }
}
```

This way, an instance of `Deck` can be supplied anywhere an `Iterable` interface type is expected. Figure 3.3 shows the main elements of the iterator design so far.

Fig. 3.3 Class diagram for the design of an iterator



One of the main ways to use `Iterable` objects in Java is in an *enhanced for loop*, also known as a *foreach* loop:

```
List<String> list = ...;

for (String string : list) {
    System.out.println(string);
}
```

The above code is just syntactic sugar for:

```
List<String> list = ...;

for (Iterator<String> iter = list.iterator(); iter.hasNext();) {
    String string = iter.next();
    System.out.println(string);
}
```

To iterate over a deck, we can now do:

```
for (Card card : deck) {
    System.out.println(card);
}
```

The way the enhanced `for` loop works is that, under the covers, it expects the rightmost part of the loop head to be an instance of a class that is a subtype of `Iterable` (or an array type, which is a special case).

The final issue to solve to complete our iterator-based design for `Deck` is to find a way to return an instance of `Iterator` when the `iterator()` method is called. Although it would be possible to hand-craft our own user-defined class that implements the `Iterator<Card>` interface, we can observe that the `List` contained within a `Deck` is also `Iterable`, and the `Iterator` it returns does everything that we want.

```
public class Deck implements Iterable<Card> {
    private List<Card> aCards;

    public Iterator<Card> iterator() {
        return aCards.iterator();
    }
}
```

Strictly speaking, this idiom can violate the encapsulation of class `Deck` because interface `Iterator<T>` includes a method `remove()` that can be optionally implemented (and which is implemented by the iterator returned by `ArrayList`). Consistent with the book's goal of focusing on general design concerns with minimum coverage of the libraries, I overlook this case. In the context of the book, it can be assumed that `Iterator#remove()` is not used. For production code, how to best avoid the encapsulation problem would depend on the context. One option is to return the iterator obtained from an unmodifiable view of the list with a call such as:

```
return Collections.unmodifiableList(aCards).iterator();
```

Representing the implementation of the `Iterator` interface in the UML is not obvious given our reliance on an instance of an unknown concrete type. Although we know that the type returned by `List#iterator()` is some subtype of the `Iterator` interface, we do not know the name of the class that ultimately implements this interface. In fact, this may be an anonymous class, in which case, by definition, there is no name. So the fact to reckon with at this point is that we do not know the type. However, we cannot really write *Unknown* in a class in a class diagram, because this would indicate that the name of the class is known, and it is "*Unknown*", which is confusing. For now, I will assume that the type is anonymous and indicate this fact with the UML *stereotype* «anonymous», leaving the name blank. In the UML, a stereotype is a variation on an element type, with the name of the variation placed in French quotes.

3.6 The ITERATOR Design Pattern

The previous section introduced the use of iterators as a way to provide access to a collection of objects encapsulated within another object without violating the in-

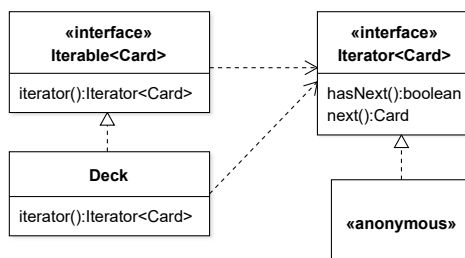


Fig. 3.4 The complete iterator design

formation hiding properties of this object. This solution is a common design pattern called, not surprisingly, the `ITERATOR`. The *context* for `ITERATOR` is to

Provide a way to access the elements of an aggregate object sequentially without exposing its underlying representation. [7]

The *solution template* for `ITERATOR` can be best captured by the class diagram in Figure 3.5, which is an abstraction of the solution presented in Figure 3.4.

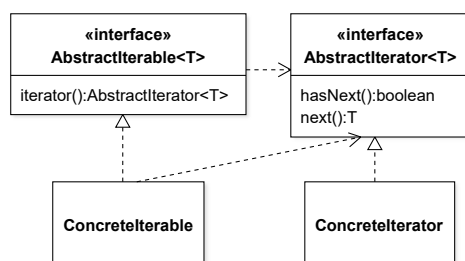


Fig. 3.5 Solution template for `ITERATOR`

An important difference between the solution template in Figure 3.5 and its concrete realization in Figure 3.4 is that the solution template does not refer to the Java library interfaces `Iterable` and `Iterator`. Although they can be supported by libraries, design patterns are abstract solution elements that can be realized in code independently of specific library implementations. For this reason, the solution template simply indicates that to apply the `ITERATOR` pattern in practice, one needs a type to fulfill the *roles* of `AbstractIterable` and `AbstractIterator`, and similarly with their concrete implementation. This being said, in Java there are very few variants of `ITERATOR`. Because only subtypes of `Iterable` can be used in the enhanced `for` loop, there is a strong incentive to actually use `Iterable` as the `AbstractIterable`, which forces the use of `Iterator` as the `AbstractIterator`. The remainder of the mapping is almost automatic, as the `ConcreteIterable` is whatever one wants to iterate over, e.g., the `Deck` class.

How to create the `ConcreteIterator` is one design decision that yields more possibilities, but the most convenient option is often to simply return the iterator supplied by the underlying data structure (e.g., `ArrayList#iterator()`), possibly wrapped in an immutable view (see Section 3.5). In some cases, it might be necessary to combine elements from various collections and iterate over them, or iterate in an order different from that of the collection holding the elements to iterate over. In such cases, the simplest option is often to create a new collection with the desired elements in the right order, and to return that collection's iterator.

Code Exploration: Solitaire · CardStack

A basic application of the ITERATOR pattern.

Class `CardStack` illustrates the simplest possible application of the `ITERATOR` in Java. The class represents a stack of cards, which I made it possible to iterate over from bottom to top. As expected, the class implements interface `Iterable<Card>` and declares a method `iterator()`. To supply the iterator, I return the iterator of an immutable view of the underlying collection. In the class, an `Iterable<Card>` is also used to define the type of the argument to one of the constructor: `CardStack(Iterable<Card>)`. This decision maximizes the flexibility of the constructor, by allowing it to take as input any type of object that can be iterated over for `Card` objects.

3.7 The STRATEGY Design Pattern

One of the major benefits of interfaces and polymorphism is to promote flexible designs. One example of a flexible design enabled by interfaces is the use of a `Comparator` instance by the `Collections.sort(...)` method, as introduced in Section 3.4. The use of function objects such as comparators to customize the behavior of another part of the code (e.g., the sorting behavior) is recognized as one application of a more general idea called the `STRATEGY` design pattern. The *context* for `STRATEGY` is to:

Define a family of algorithms, encapsulate each one, and make them interchangeable. Strategy lets the algorithms vary independently from clients that use it. [7]

This is a very general definition, especially given that there is no agreed-upon definition for what a *family of algorithms* is. Fortunately, the *solution template* for `STRATEGY` provides a clarification for object-oriented code: algorithms in the same family implement the same interface.

The `STRATEGY` looks exceedingly simple. In fact in many cases it can be indistinguishable from a basic use of polymorphism. I find it useful to think of a part of the design as an application of `STRATEGY` when that part of the design is focused on allowing the switch between algorithms. One example, illustrated in Figure 3.7, is

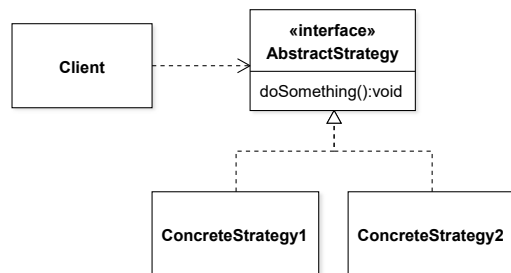


Fig. 3.6 Solution template for STRATEGY

the use of different card comparators for sorting a deck of cards. Another example is the implementation of different automatic playing strategies, as will be further discussed in the Code Exploration section below.

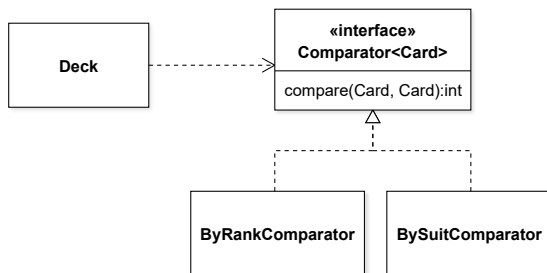


Fig. 3.7 Sample instantiation of a STRATEGY

Although nominally simple, in practice applying a STRATEGY requires thinking about many design questions:

- Does the `AbstractStrategy` need one or multiple methods to define the algorithm? Typically the answer is one, but in some cases it may be necessary to have more methods.
- Should the strategy method return anything or have a side-effect on the argument?
- Does a strategy need to store data?
- What should be the type of the return value and/or method parameters, as applicable? Ideally we want to choose these types to minimize coupling between a strategy and its clients.

Figure 3.7 shows an example instantiation of STRATEGY for the context of comparing cards. Here the design of the `AbstractStrategy` is already decided because we are reusing the `Comparator` interface. This strategy is purely functional as it

does not have any side-effect and returns the result of applying the comparison algorithm. At this point it should become clearer that implementing the `Comparator` interface as a `UniversalComparator` that holds a value to decide what kind of comparison to do, does not respect the spirit of the STRATEGY because the actual strategy would be selected by changing the state of an object, as opposed to changing the concrete strategy object.

Code Exploration: Solitaire · PlayingStrategy

Applying the STRATEGY to implement game-playing strategies.

In Solitaire, the package `auto` provides an elaborate example of the STRATEGY pattern. Interface `PlayingStrategy` defines a method `getLegalMove` that is called by a `GameModel` instance when the player uses the *auto-play* feature. With the auto-play feature, the software makes a decision of how to play the next move in the game, as opposed to waiting for the user to play a move. Any class that implements `PlayingStrategy` can provide a decision-making behavior for making a move. The package contains two examples of strategies. The `NullPlayingStrategy` never does anything, and always returns a so-called *null move*. The purpose of this code is explained in Section 4.4. The `GreedyPlayingStrategy` selects the move with the most immediate impact on the game. To implement a different strategy, for example one that uses probabilities in the decision-making process, we define a new class that implements `PlayingStrategy` and use an instance of this new class in the `GameModel`.

3.8 Dependency Injection

So far, we have seen how to use interface types so that classes that use a service can be decoupled from the actual implementation of the service. Continuing with our example of the comparator, let us say we are designing a version of our `Deck` of cards that can be sorted in various ways. Applying the STRATEGY pattern described in the previous section, our code should look like this:

```
public class Deck {
    private List<Card> aCards = new ArrayList<>();
    private Comparator<Card> aComparator = /* initialize */;

    public void sort() {
        Collections.sort(aCards, aComparator);
    }
}
```

In the code above, various options are possible for initializing the comparator (that is, the concrete strategy). One option is to call the constructor of the desired comparator when initializing the field:

```
private Comparator<Card> aComparator = new ByRankComparator();
```

There are two issues with this approach. First, it does not allow the client code to easily switch the comparison strategy. To switch the comparison strategy, it would be necessary to modify the source code of the `Deck` class. Second, this design introduces a *dependency* between the `Deck` class and a specific comparator implementation. Figure 3.8 illustrates the problem.

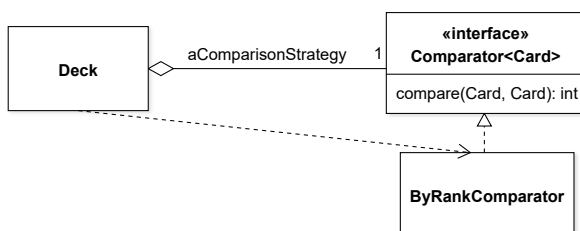


Fig. 3.8 Introducing a dependency between a client class and an implementation class

One variant of this solution could be to use an anonymous class in the definition of the comparator:

```
private Comparator<Card> aComparator = new Comparator<Card>() {
    public int compare(Card pCard1, Card pCard2) {
        return pCard1.getRank().compareTo(pCard2.getRank());
    }
};
```

However, this solution exhibits the same problem as the previous one: it does not allow us to switch the comparison algorithm easily, and the `Deck` class is still coupled to a specific implementation of the comparison. The only difference is that now this implementation is anonymous.

A solution to both the lack of flexibility of the `Deck` class and its tight coupling with the comparison strategy is to decouple the creation of the dependency (here, the implementation of the comparator) from the creation of the client of the dependency (here, the `Deck` class). Instead, we pass in, or *inject*, the dependency into the client class.

```
public class Deck {
    private Comparator<Card> aComparator;

    public Deck(Comparator<Card> pComparator) {
        aComparator = pComparator;
        shuffle();
    }
}
```

This technique is called *dependency injection*. In this way, class `Deck` only has a dependency to the interface type `Comparator<Card>`, and remains decoupled from

any specific implementation. The trade-off is that client code of the `Deck` class must now inject this dependency when creating a `Deck`:

```
Deck deck = new Deck(new ByRankComparator());
```

As is the case for design patterns, dependency injection is a general idea and there are many different ways to apply it in practice. For example, one could design various factory methods to instantiate dependency objects, or inject a dependency using an anonymous class, etc. It is also possible to inject the dependency via a setter method instead of the constructor. This alternative, however, is often inferior because it creates object state management challenges, discussed in Chapter 4. Finally, there are also libraries and frameworks available to support advanced dependency injection scenarios that require a lot of configuration. In this book, however, I stick to simple applications of dependency injection such as the one illustrated above.

Code Exploration: Solitaire · GameModel

Injecting a concrete strategy into another object.

The concrete `PlayingStrategy` described in the previous Code Exploration is injected into the `GameModel` via its constructor. The application is assembled in method `start()` of class `Solitaire`. In this method, we see the creation of the concrete strategy and its injection into the `GameModel`:

```
GameModel model = new GameModel(new GreedyPlayingStrategy());
```

3.9 The Interface Segregation Principle

Throughout this chapter we saw the various benefits of defining specialized interfaces that specify a small and coherent slice of behavior that clients depend on. This way, client code is not coupled with the details of an implementation, and only depends on the methods it actually requires. For example, code that processes cards can only depend on a `CardSource` interface with two methods, and can therefore be reusable with any class that can provide these methods. Similarly, the `Collections#sort(...)` method works because it can rely on just the fact that the items in the collection are `Comparable`. This idea is actually an instance of a general design principle called the *Interface Segregation Principle (ISP)*. Simply put, the ISP states that *client code should not be forced to depend on interfaces it does not need*.

The idea of the ISP is easier to explain by presenting a situation where the principle is *not* respected. We can consider again the code in Section 3.1 where `drawCards` takes a `Deck` as argument:

```
public static List<Card> drawCards(Deck pDeck, int pNumber) {
    List<Card> result = new ArrayList<>();

    for (int i = 0; i < pNumber && !pDeck.isEmpty(); i++) {
        result.add(pDeck.draw());
    }
    return result;
}
```

In Section 3.1 I argued that this was a suboptimal design because it tied an interface with its implementation. Well, let us say we split the two by declaring an `IDeck` interface:

```
public interface IDeck {
    void shuffle();
    Card draw();
    boolean isEmpty();
}

public class Deck implements IDeck...
```

which we rely on in `drawCards`:

```
public static List<Card> drawCards(IDeck pDeck, int pNumber)
```

This effectively decouples interface from implementation, and supports the use of `drawCards` with arguments that are not strictly instances of `Deck`. However, this design also forces `drawCards` to statically depend on a method it does not need, namely, `shuffle`. What if we might want to draw cards from a source that cannot be shuffled? For this reason, the `CardSource` solution initially presented in Section 3.1 only included methods `draw` and `isEmpty` in interface `CardSource`, and thereby respected the ISP.

To push on this idea of ISP a bit, let us assume that there might be places in the code that only shuffle an object. To support this slice of behavior, we would define an interface `Shuffleable` with a single method `shuffle()`. Figure 3.9 shows a maximally flexible separation of concerns for class `Deck`, with three different interfaces that capture three cohesive slices of behavior that are supported by class `Deck`, and three client code locations (represented by `Client1-3`) interested in different combinations of these services.

This design has loose coupling, which is great. However, this loose coupling has one major disadvantage for cases where a client might be interested in more than one slice of behavior. This situation is represented in Figure 3.9 by `Client3`, which needs to both iterate over an `Iterable<Card>` *and* draw some cards from the source. How can we express this combination, since in Java it is only possible to specify a single type for a variable? For example, if we pass an instance of `CardSource` to a method of interest and wish to iterate over the cards, we have to venture into inelegance:

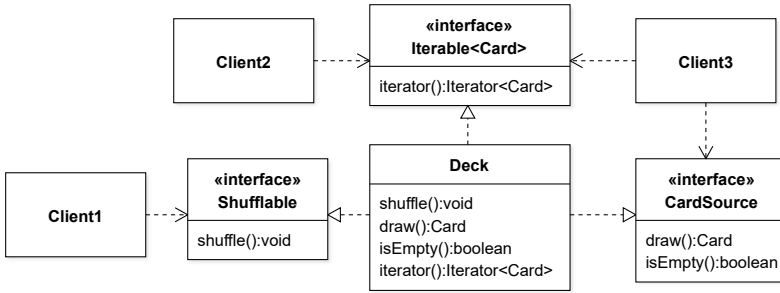


Fig. 3.9 Interface segregation in practice

```
public void displayCards(CardSource pSource) {
    if (!pSource.isEmpty()) {
        pSource.draw();
        for (Card card : (Iterable<Card>) pSource) {
            ...
        }
    }
}
```

In fact this is not only inelegant, but also unsafe, because it could be possible to provide an argument to `displayCards` that is *not* a subtype of `Iterable`. A better solution to this issue is offered directly by the type system, in the form of subtyping. In Java, interfaces can be declared to extend each other, with the semantics that if `A` extends `B`, types that implement `A` must provide implementations for all the methods declared in `B` as well, transitively. By extending interfaces, we can more easily support combinations of services while respecting the ISP. In our scenario, if it is observed that a lot of the code that uses `CardSource` also uses `Iterable<Card>`, *but not the other way around*, then we can declare `CardSource` to be a subtype of `Iterable<Card>`, as illustrated in Figure 3.10.

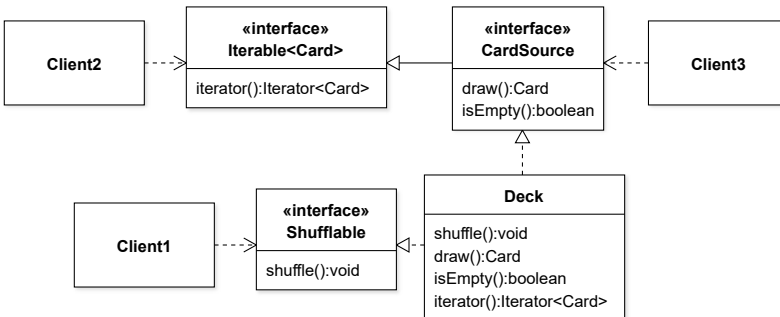


Fig. 3.10 Example of interface extension

In principle, the same reasoning could apply in the reverse situation: if we notice that most code that uses `Iterable<Card>` also uses `CardSource` *but not the other way around*, it would make sense to declare `Iterable<Card>` to extend `CardSource`. In practice, however, this is not possible because `Iterable` is a library type, which it is not possible to modify. For this reason, there are also large amounts of code that depend on it without depending on `CardSource`, rendering the situation a mere theoretical possibility. Finally, software designs can be hard to get right immediately. It may be the case that two segregated interfaces end up always being used together in the client code. In such a situation, it is possible that application of the ISP went too far, and it might be worth considering fusing two interfaces back into one, by collecting all method declarations into a single interface.

Code Exploration: Solitaire · GameModel

Using the Interface Segregation Principle when applying the STRATEGY.

In Solitaire, the STRATEGY is used via interface `PlayingStrategy`, which defines a method `getLegalMove`. To decide how to make a move, a strategy object needs some information about the game. This information is provided as a parameter to the strategy method. All the information about an on-going game is stored in an instance of `GameModel`. This is a large class which declares both accessor methods and methods that can change the state of the game. Hence, by passing an instance of `GameModel` to a strategy object, the code of the strategy object would be able to modify the state of the game. This is excessive coupling, because in the design the playing strategies are only supposed to compute a move, not actually do the move. To make this constraint clearer, I used the Interface Segregation Principle as follows. Class `GameModel` implements an interface `GameModelView` that declares *only* the methods of `GameModel` that provide information about the game without changing anything. The effect of this decision is to have a type that *narrows* the interface of `GameModel` to only include query methods. I then declared the type of the parameter of `getLegalMove` to be `GameModelView`. This way, although it is still actually an instance of `GameModel` that is provided to strategy objects, the limited interface makes it clear that the code of strategy objects is only meant to query the state of the game, not change it.

Code Exploration: JetUML · DiagramElement

Organizing interfaces into a hierarchy to serve different usage contexts.

A diagram in JetUML contains different diagram elements that are either nodes or edges. I used interface types `Node` and `Edge` to specify the expected behavior of these elements. However, certain parts of the code, like the `Clipboard` class, need to deal with diagram elements in general, independently of whether they are nodes or edges. To accommodate this flexibility, I defined an additional interface, `DiagramElement`, which is extended by both

Node and Edge. The behavior for general elements, nodes specifically, and edges specifically, is thus segregated into different interface types. This allows the various client code locations to work with the most appropriate abstraction for the diagram element objects they need to handle.

Insights

This chapter focused on how to use interfaces and polymorphism to achieve extensibility and reuse.

- Use interface types to decouple a specification from its implementation if you plan to have different implementations of that specification as part of your design;
- Define interface types so that each type groups a cohesive set of methods that are likely to be used together;
- Organize interface types as subtypes of each other to create flexible groupings of behavior;
- Use library interface types, such as `Comparable<T>`, to implement commonly expected behavior;
- Use class diagrams to explore or capture important design decisions that have to do with how classes relate to each other;
- Consider function objects as a potential way to implement a small piece of required functionality, such as a comparison algorithm. Function objects can often be specified as instances of anonymous classes, or as lambda expressions;
- Use iterators to expose a collection of objects encapsulated within another without violating the encapsulation and information hiding properties of this object. This idea is known as the `Iterator` design pattern;
- Consider using the `Strategy` pattern if part of your design requires supporting an interchangeable family of algorithms;
- Use dependency injection to decouple a client class that uses some abstraction from the creation of this abstraction;
- Ensure that your code does not depend on interfaces it does not need: break up large interface types into smaller ones if you find that many methods of a type are not used in certain code locations.

Further Reading

The definitions for the notation of the class diagram shown in Figure 3.1 are adapted from *The Unified Modeling Language Reference Manual* [15]. Chapter 3 of *UML Distilled* [6] provides an overview of the notation and semantics for this diagram. The Gang of Four book [7] has the original treatment of the `Iterator` and `Strategy` patterns.